# Artificial Intelligence

AI-2002

# Lab 02

OOP in Python

| Course Code: AI-2002 | Artificial Intelligence Lab |
|---|---|

1. **Objective** ..... 3

2. **Introduction to Object-Oriented Programming (OOP)** ..... 3

2.1 Class..... 4
2.2 Object ..... 5
2.3 Attributes and Methods ..... 6
2.4 Constructor in Python ..... 8
2.5 Types of Attributes in a Class ..... 10
2.5.1 Class Attributes.....10
2.5.2 Instance Attributes.....11
2.6 Practice Task #1 (Smart Light System) ..... 14

3. **Inheritance in Python ..... 15**

3.1 Basic Concept of Inheritance..... 15
3.2 Inheritance without super() ..... 16
3.3 Inheritance with super() ..... 17
3.4 Types of Inheritance..... 18
3.4.1 Single Inheritance.....18
3.4.2 Multiple Inheritance.....18
3.4.3 Multilevel Inheritance.....19
3.4.4 Hierarchical Inheritance .....19
3.4.5 Hybrid Inheritance.....19
3.5 Practice Task #2 (University Staff System) ..... 20

4. **Polymorphism in Python ..... 21**

4.1 Compile-Time Polymorphism (Simulated)..... 21
4.1.1 Method Overloading (Default Arguments) .....21
4.1.2 Method Overloading (*args) .....22
4.2 Run-Time Polymorphism ..... 23
4.2.1 Method Overriding ..... 23

5. **Encapsulation in Python ..... 24**

5.1 Public Members..... 24
5.2 Protected Members ..... 25
5.3 Private Members ..... 26
5.4 Practice Task #3 (Bank Account System)..... 27

6. **Lab Tasks ..... 20**

# 1. Objective

1. Introduction to Object-Oriented Programming (OOP) concepts using Python.
2. To develop basic OOP programming skills in Python using classes and objects.
3. To implement core OOP principles such as encapsulation, inheritance, and polymorphism.
4. To solve basic programming problems using OOP in Python

# 2. Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using classes and objects to model real-world entities. In Python, OOP allows programmers to create reusable and modular code by encapsulating data and functions within classes. Key concepts of OOP include encapsulation (hiding internal details), inheritance (reusing code from existing classes), polymorphism (using the same interface for different objects), and abstraction (focusing on essential features while hiding complexity). Learning OOP in Python helps students design structured programs, improve problem-solving skills, and develop software that is easier to maintain and extend.

## 2.1 Class

A class is a blueprint or template for creating objects. It defines the attributes (data) and methods (functions) that the objects created from it will have. Think of it as a plan for building something, like a blueprint for a house.

### How to Create a Class in Python

```
class Student:
pass
```

## 2.2 Object

An object is an instance of a class. It represents a real entity created using the class blueprint, with its own values for the attributes.

### How to Call and Object in Python

```
s1 = Student()    # s1 is an object of class Student s2
= Student()   # s2 is another object
```

## 2.3 Attributes and Methods

In Python, **attributes** are variables that belong to a class and hold the data or properties of an object. **Methods** are functions defined inside a class that describe the behaviors or actions of an object. Attributes store information specific to each object, while methods allow objects to perform operations using that data. Using attributes and methods together helps in organizing code in a structured and reusable way, making programs easier to understand and maintain.

### Creating/Declaring Attributes

```
class MyClass:    x = 5 #x
is an attribute
```

### Creating/Declaring Method

```
class MyClass:
  x = 5 #x is an attribute
     #Method    def
method_one(self):
    pass
```

### Accessing Attributes and Methods

```
print(m1.x) m1.method_one()
```

### Full Code

```
class MyClass:
  x = 5 #x is an attribute
   #Method   def
method_one(self):
    print("This is method one")

m1= MyClass()  #m1 is attribut
print(m1.x) m1.method_one()
```

## 2.4 Constructor in Python

A constructor is a special method in a class that is automatically called when an object is created. It is used to initialize the attributes of an object. In Python, the constructor method is always named __init__().

### Creating and Accessing Constructor

```
class Student:     def
__init__(self, name, age):
        self.name = name  # instance attribute
self.age = age    # instance attribute

# Creating objects s1 =
Student("Ali", 20) s2 =
Student("Sara", 22)
 print(s1.name, s1.age)  # Output: Ali 20
print(s2.name, s2.age)  # Output: Sara
22
```

## 2.5 Types of Attributes in a Class

In Python, attributes are variables that belong to a class. They are used to store data related to **objects**. There are **two main types of attributes**:

1. Class Attribute
2. Instance Attribute

## Class Attribute

- Defined directly inside the class, outside any method.
- Shared by all instances of the class.

---

```
class Student:
    school = "ABC High School"  # class attribute
 s1 = Student() s2 = Student()
print(s1.school)  # ABC High
School print(s2.school)  # ABC
High School
```

---

## Instance Attribute

- Defined inside the __init__ constructor using self.
- Each object (instance) has its own copy of these attributes.

---

```
class Student:     def
__init__(self, name, age):
        self.name = name   # instance attribute
self.age = age     # instance attribute
 s1 = Student("Ali", 20) s2 =
Student("Sara", 22)
print(s1.name, s1.age)  # Ali 20
print(s2.name, s2.age)  # Sara
22
```

---

## Modifying Class and Instance Attribute =

---

```
class Student:
    school_name = "ABC School"  # class attribute (shared by all
students)      def __init__(self, name, age):
        self.name = name   # instance attribute
self.age = age     # instance attribute
     def
display_info(self):
        print(f"Name: {self.name}")
print(f"Age: {self.age}")
print(f"School: {Student.school_name}")
```

```
# Creating student objects (instances)
student1 = Student("Ali", 15)
student2 = Student("Sara", 16)

# Displaying info student1.display_info()
print("-----------------")
student2.display_info()

# Modifying an instance attribute student1.age
= 16
print("\nAfter updating age of student1:")
student1.display_info()
```

---

---

```
# Modifying the class attribute
Student.school_name = "XYZ School"
print("\nAfter updating school name:")
student1.display_info()
student2.display_info()
```

---

## Example

Suppose you are asked to create a program to **manage bank accounts**. Each account has a **holder's name, account number, and balance**, and can perform actions like viewing account info and depositing money.

This scenario demonstrates the importance of OOP in Python:

- **Modularity:** Code is organized into a class BankAccount, separating account logic from other code.
- **Reusability:** Multiple accounts can be created from the same class without rewriting code.

---

```
class BankAccount:
    bank_name = "ABC Bank"  # class attribute shared by all accounts
     def __init__(self, holder_name, account_no, balance):
self.holder_name = holder_name    # instance attribute
self.account_no = account_no      # instance attribute
self.balance = balance            # instance attribute
     def display_account(self):            # method to display
account info          print("Bank:", BankAccount.bank_name)
print("Account Holder:", self.holder_name)          print("Account
Number:", self.account_no)          print("Balance:", self.balance)
```

```
    def deposit(self, amount):            # method to deposit money
self.balance += amount
        print(f"Deposited {amount}. New Balance: {self.balance}")

#Create Objects / Instances account1 =
BankAccount("Ali", 1001, 5000) account2 =
BankAccount("Sara", 1002, 7000)

#Call Methods
account1.display_account()
```

---

```
print("----------------") account2.display_account()
print("----------------")
account1.deposit(2000)
```

---

## 2.6 Practice Task # 1

Create a program to manage parking slots in a multi-level parking plaza. Each parking slot has a slot number and a status (OCCUPIED/VACANT). The system should allow:

1. Creating multiple parking slot objects for different slots.
2. Marking slots as OCCUPIED or VACANT.
3. Displaying the current status of each slot.

**Objective:**
This task demonstrates **OOP in AI automation**:

- **Classes** organize devices logically.
- **Objects** represent individual parking slots.
- **Methods** automate actions like marking slots OCCUPIED/VACANT.

### *Expected Output*

```
Slot A1 marked as OCCUPIED.
Slot B3 marked as VACANT.
Slot A1 is OCCUPIED.
Slot B3 is VACANT.
```

---

## 3. Inheritance in Python

Inheritance is a feature of Object-Oriented Programming (OOP) that allows a class (called the child or subclass) to reuse the properties and methods of another class (called the parent or superclass). This helps in code reusability and reduces redundancy.

- Parent Class (Superclass): The class whose properties and methods are inherited.
- Child Class (Subclass): The class that inherits properties and methods from the parent class.
- Child classes can add new methods or override existing ones from the parent class.

### Inheritance Basic Syntax

```
class ParentClass:
    # parent class code
 class
ChildClass(ParentClass):
    # child class code
```

### Inheritance without Super

```
class ParentClass:
def p(self):
        print('parent func')
 class
ChildClass(ParentClass):
def c(self):
        print('child func')
self.p()    # parent function call
 co = ChildClass() co.c()    # child
func + parent func

# Overriding
```

```
#Accessing attributes of parent class without Super
class Parent:     def __init__(self):
self.x = 10
 class Child(Parent):
def __init__(self):
self.y = 20
 c = Child()
print(c.x)    # ✗
error
```

---

## Inheritance with Super ()

---

```
class Parent:
def p(self):
        print("parent func")
 class
Child(Parent):
def p(self):
        print('child func')
     def c(self):
self.p()
super().p() co =
Child() co.c()

#Accessing attributes of parent class without Super
class Parent:     def __init__(self):
self.x = 10
 class Child(Parent):
def __init__(self):
self.y = 20
 c = Child()
print(c.x)    # ✗
error
```

---

## 3.1 Types of Inheritance

1. **Single Inheritance:** A child class inherits from a single parent class.
2. **Multiple Inheritance:** A child class inherits from more than one parent class.
3. **Multilevel Inheritance:** A child class inherits from a parent class, which in turn inherits from another class.
4. **Hierarchical Inheritance:** Multiple child classes inherit from a single parent class.
5. **Hybrid Inheritance:** A combination of two or more types of inheritance.

## Single Inheritance

---

```
# Parent class class
Parent:     def
greet(self):
        print("Hello from Parent")
 # Child class class
Child(Parent):     def
greet_child(self):
        print("Hello from Child")
 c =
Child()
c.greet()         # inherited from Parent
c.greet_child()   # child method
```

---

## Multiple Inheritance

---

```
class Parent1:     def
greet_parent1(self):
        print("Hello from Parent1")
 class Parent2:     def
greet_parent2(self):
        print("Hello from Parent2")
 class Child(Parent1,
Parent2):     def
greet_child(self):
        print("Hello from Child")
 c =
Child()
c.greet_parent1()
```

```
c.greet_parent2()
c.greet_child()
```

---

## MultiLevel Inheritance

---

```
# Grandparent class class
Grandparent:     def
greet_grandparent(self):
        print("Hello from Grandparent")

# Parent class class
Parent(Grandparent):
def greet_parent(self):
        print("Hello from Parent")

# Child class class
Child(Parent):     def
greet_child(self):
        print("Hello from Child")
 c =
Child()
c.greet_grandparent()  # inherited from Grandparent
c.greet_parent()       # inherited from Parent
c.greet_child()        # own method
```

---

## Hierarchal Inheritance

---

```
class Parent:
    def greet_parent(self):
        print("Hello from Parent")
 class Child1(Parent):
def greet_child1(self):
        print("Hello from Child1")
 class
Child2(Parent):
    def greet_child2(self):
        print("Hello from Child2")
 c1 = Child1() c2
= Child2()
```

```
c1.greet_parent()
c1.greet_child1()
```

---

## Hybrid Inheritance

---

```
class Grandparent:     def
greet_grandparent(self):
        print("Hello from Grandparent")
 class Parent1(Grandparent):
def greet_parent1(self):
        print("Hello from Parent1")
 class Parent2:     def
greet_parent2(self):
        print("Hello from Parent2")
 class Child(Parent1,
Parent2):     def
greet_child(self):
        print("Hello from Child")
 c =
Child()
c.greet_grandparent()
c.greet_parent1()
c.greet_parent2()
c.greet_child()
```

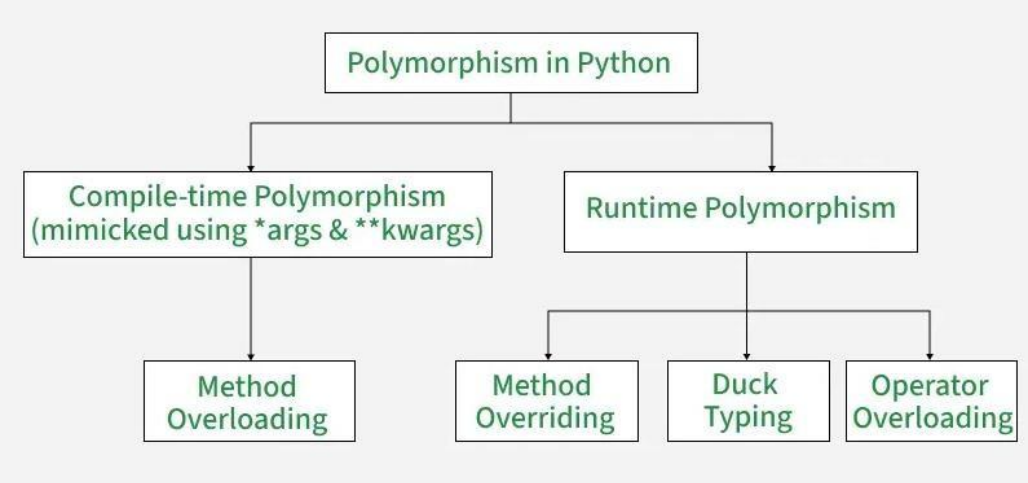
---

## Practice Task # 2

You are designing a Hospital Staff Management System. The system needs to keep track of different hospital staff members: Doctors, Nurses, and Lab Technicians. All staff share common attributes like name, employee_id, and department, but each type has its own specific properties and behaviors. Doctors have specialization and consultation_fee and can examine patients, Nurses have shift_timing and ward_assigned and can administer medication, while Lab Technicians have lab_section and equipment_handled and can conduct tests. Your task is to create a base class HospitalStaff and derive classes for each staff type, instantiate objects, and call their respective methods to simulate daily activities. Optionally, override the display_info() method to show type-specific details.

## 4. Polymorphism in Python

Polymorphism means “many forms”. In Python OOP, it allows different objects or methods to respond differently to the same action.




## 4.1. Compile Time Polymorphism

This type of polymorphism is determined during the compilation of the program. It allows methods or operators with the same name to behave differently based on their input parameters or usage. In languages like Java or C++, compile-time polymorphism is achieved through method overloading but it's not directly supported in Python.

In Python:

- True compile-time polymorphism is not supported.
- Instead, Python mimics it using default arguments or *args/**kwargs.
- Operator overloading can also be seen as part of polymorphism, though it is implemented at runtime in Python

---

```
# ---------- Method Overloading (Simulated) using Default Argument
class Calculator:     def add(self, a, b=0):
        """Add two numbers, or one number if only one is given"""
print("Sum:", a + b)
 calc =
Calculator()
print("Method Overloading Examples:") calc.add(5,
10)  # Using two parameters calc.add(7)      # Using
one parameter (default b=0)

# ---------- Method Overloading (Simulated) using Kwargs Argument
class Calculator:     def add(self, *args):         return
sum(args)
 calc = Calculator() print(calc.add(5, 10))
# Two arguments
```

---

```
print(calc.add(5, 10, 15))    # Three arguments
print(calc.add(1, 2, 3, 4))   # Any number of arguments
```

## 4.2. Run Time Polymorphism

Run-Time Polymorphism is determined during the execution of the program. It covers multiple forms in Python:

1. **Method Overriding:** A subclass redefines a method from its parent class.
2. **Duck Typing:** If an object implements the required method, it works regardless of its type.
3. **Operator Overloading:** Special methods (__add__, __sub__, etc.) redefine how operators behave for user-defined objects.

Example: In this example, we show run-time polymorphism using method overriding with dog classes and compile-time polymorphism by mimicking method overloading in a calculator class.

## Method Overriding

```
# # Parent class
class Animal:
def sound(self):
        print("Some generic sound")

# Child class class
Dog(Animal):     def
sound(self):
        print("Bark")   # overrides parent method
 class Cat(Animal):
def sound(self):
        print("Meow")   # overrides parent method

# Runtime polymorphism
a = Dog() b = Cat()
a.sound()   # Bark
b.sound()   # Meow
```

## 4.6 Encapsulation in Python

Encapsulation is an OOP principle that restricts direct access to the internal data of a class and bundles the data (attributes) and methods that operate on that data together. It helps to:

- Protect sensitive data
- Control how attributes are accessed or modified
- Make code more secure and maintainable

In Python, encapsulation is implemented by using private attributes, which are prefixed with a single _ (protected) or double __ (private).

## Types of Encapsulation:

1. **Public Members**: Accessible from anywhere.
2. **Protected Members**: Accessible within the class and its subclasses.
3. **Private Members**: Accessible only within the class.

## Private Methods and Attributes

```
class Student:     def __init__(self,
name, roll_no):
        self.name = name          # public attribute
self.roll_no = roll_no    # public attribute

    # public method     def
display_info(self):
        print(f"Name: {self.name}, Roll No: {self.roll_no}")

# Accessing within the same class s
= Student("Ali", 101)
print("Accessing Public Attribute and Method in same class:")
print(s.name)           # public attribute print(s.roll_no)
s.display_info()        # public method

# Accessing from another class (subclass)
class CollegeStudent(Student):     def
show(self):
```

```
        print("\nAccessing Public Attribute and Method from
subclass:")          print(self.name)
print(self.roll_no)          self.display_info()
 cs = CollegeStudent("Sara",
102) cs.show()
```

## Protected Methods and Attributes

```
class Student:      def __init__(self,
name, roll_no):
        self._name = name         # protected attribute
self._roll_no = roll_no  # protected attribute

    # protected method
def _display_info(self):
        print(f"Name: {self._name}, Roll No: {self._roll_no}")

# Accessing within the same class s = Student("Ali", 101)
print("\nAccessing Protected Attribute and Method in same class:")
print(s._name) print(s._roll_no) s._display_info()

# Accessing from another class (subclass)
class CollegeStudent(Student):      def
show(self):
        print("\nAccessing Protected Attribute and Method from
subclass:")          print(self._name)
print(self._roll_no)          self._display_info()
 cs = CollegeStudent("Sara",
102) cs.show()
```

## Private Methods and Attributes

```
class Student:      def __init__(self,
name, roll_no):
        self.__name = name         # private attribute
self.__roll_no = roll_no  # private attribute
```

```
    # private method     def
__display_info(self):
        print(f"Name: {self.__name}, Roll No: {self.__roll_no}")

    # public method to access private members     def
access_private(self):         print("Accessing private members
inside the same class:")         print(self.__name)
print(self.__roll_no)         self.__display_info()

# Accessing private attribute and method **inside the same class**
via public method s = Student("Ali", 101)
s.access_private()

# Accessing from another class (subclass) class
CollegeStudent(Student):
    def show(self):
        print("\nTrying to access private members from subclass:")
        # The following will cause error if uncommented:
        # print(self.__name)
        # self.__display_info()
        # Correct way: use a public method
self.access_private()

cs = CollegeStudent("Sara", 102) cs.show()
```

---

# Practice Task # 3

You are designing a simple inventory management system for a warehouse, where each product has a stock quantity. The stock quantity should be private and not directly accessible from outside the class. Your task is to create a Product class with a private attribute __stock_quantity and public methods to add_stock, remove_stock, and get_stock. Create a few product objects and use these methods to perform stock updates, observing how encapsulation protects the stock quantity from direct access.

## Maths Module

---

```
import math
 print("pi:",
math.pi) print("e:",
math.e)
```

```
# Basic functions print("ceil(4.2):", math.ceil(4.2))        #
nearest upper integer print("floor(4.8):", math.floor(4.8))      #
nearest lower integer print("fabs(-7.5):", math.fabs(-7.5))      #
absolute float print("factorial(5):", math.factorial(5))  # 5! =
120 print("sqrt(16):", math.sqrt(16))          # square root
print("pow(2,3):", math.pow(2,3))          # 2^3 = 8.0
print("exp(2):", math.exp(2))              # e^2
print("log(8,2):", math.log(8,2))          # log base 2
print("log10(100):", math.log10(100))      # log base 10
print("log2(16):", math.log2(16))          # log base 2
print("isnan(math.nan):", math.isnan(math.nan))  # check NaN
print("isinf(math.inf):", math.isinf(math.inf))  # check infinity

# Trigonometric functions print("sin(pi/2):",
math.sin(math.pi/2)) print("cos(0):", math.cos(0))
print("tan(pi/4):", math.tan(math.pi/4))
print("asin(1):", math.asin(1))  # inverse sine
print("acos(1):", math.acos(1))  # inverse cosine
print("atan(1):", math.atan(1))  # inverse tangent
print("atan2(1,1):", math.atan2(1,1)) # atan(y/x)
print("degrees(pi):", math.degrees(math.pi))
print("radians(180):", math.radians(180))

# Other useful constants / functions
print("copysign(5,-1):", math.copysign(5,-1))   # 5 with sign of -1
print("fmod(10,3):", math.fmod(10,3))          # remainder like %
print("gcd(12,18):", math.gcd(12,18))          # greatest common
divisor
print("lcm(12,18):", math.lcm(12,18))          # least common
multiple (Python 3.9+)
```

---

## Random Module

---

```
import random
 print("random():", random.random())
print("uniform(1,10):", random.uniform(1, 10))
```

```
 print("randint(1,6):", random.randint(1, 6))
print("randrange(1,10):", random.randrange(1, 10))
print("randrange(1,10,2):", random.randrange(1, 10, 2))
 colors = ['red', 'blue', 'green',
'yellow'] print("choice():",
random.choice(colors))
 nums = [1, 2, 3, 4, 5] print("choices(k=3):",
random.choices(nums, k=3)) print("sample(k=3):",
random.sample(nums, 3))
 random.shuffle(nums)
print("shuffle():", nums)
 random.seed(10) print("seed + randint:",
random.randint(1, 100))
 print("gauss(0,1):", random.gauss(0, 1))
print("normalvariate(0,1):", random.normalvariate(0, 1))
print("expovariate(1.5):", random.expovariate(1.5))
print("triangular(1,10,5):", random.triangular(1, 10, 5))
```

---

# LAB TASKS

## TASK #1

You are building an Online Library Management System for a university that wants to digitize how it tracks its book inventory. The library currently manages hundreds of books manually, which makes it difficult to know which books are available and which are currently checked out. Your job is to design an object-oriented solution that models each book as an object and allows the library staff to manage borrowing and returning of books efficiently. Create a class Book that uses a constructor to initialize the instance attributes title, author, and is_available (which should default to True when a book is first added to the system), along with a class attribute library_name that is shared by all Book objects since every book belongs to the same library. Implement a method borrow_book() that checks whether the book is available; if it is, it should mark the book as unavailable and print a confirmation message, otherwise it should print a message saying the book is already borrowed, and a method return_book() that marks the book as available again and prints a confirmation message.

## TASK #2

You are designing a Vehicle Rental System for a car rental company that offers three tiers of service: regular vehicles, cars, and luxury cars with a chauffeur. The company wants a system where common vehicle information is defined once and reused across all vehicle types, while each tier adds its own additional features and pricing rules on top of the base rental cost. Create a base class Vehicle with attributes vehicle_id, brand, and rent_per_day, initialized through a constructor, along with a method calculate_rent(days) that returns the rent for the given number of days. Create a subclass Car that inherits from Vehicle and adds an additional attribute fuel_type (e.g., Petrol, Diesel, or Electric), while reusing the constructor and rent calculation logic of the parent class using super(). Then create a subclass LuxuryCar that inherits from Car, making this a multilevel inheritance hierarchy of Vehicle to Car to LuxuryCar, and adds an additional attribute chauffeur_fee.

## TASK #3

You are creating a Smart Home Appliance Controller. Create two parent classes: RemoteControllable (with method turn_on_off()) and EnergyMonitor (with method check_power_usage()). Create a class SmartAC that inherits from both (multiple inheritance) and adds its own method set_temperature(temp). Separately, create a base class Appliance with a brand attribute, and derive SmartFridge and SmartTV from it (hierarchical inheritance), each with their own specialized method. Instantiate objects and call all relevant methods.

## TASK #4

You are building a Food Delivery Billing System. Create a class Order with a method calculate_total(*items) that uses *args to sum item prices regardless of how many items are passed (simulated method overloading). Then create a base class DeliveryService with a method deliver(), and derived classes BikeDelivery, CarDelivery, and DroneDelivery that each override deliver() to print a different delivery message (method overriding). Demonstrate runtime polymorphism by looping through a list of mixed delivery objects and calling deliver() on each.

## TASK #5

You are creating a student grading system. Each student has a private attribute __grade representing their score. The class Student should have methods set_grade(grade) to update the score, get_grade() to view it, and display_info() to show the student's name and grade. Students should create student objects, update grades using setters, retrieve them using getters, and understand how encapsulation keeps the grade secure from direct modification.

## TASK #6

You are developing an AI-based E-Commerce Recommendation System for an online store that sells products across multiple categories. The store wants a unified way to manage product information and stock levels while still being able to generate category-specific recommendations for customers browsing the site, all while keeping stock data safely encapsulated so it cannot be modified incorrectly from outside the class. Create a base class Product with a constructor that initializes protected attributes _name and _price, and a private attribute __stock representing the number of units available. Implement a public method update_stock(qty) that increases or decreases __stock by the given quantity, where qty may be positive for restocking or negative for a sale, while preventing the stock from going below zero, and a public method get_stock() that returns the current value of __stock, since it cannot be accessed directly from outside the class. Create three derived classes, Electronics, Clothing, and Groceries, each overriding a method recommend() to print a category-specific recommendation message referencing the product's _name and _price. Create a list containing a mix of Electronics, Clothing, and Groceries objects, loop through the list calling recommend() on each to demonstrate polymorphism, and then demonstrate a few calls to update_stock() and get_stock() to show that the stock levels are being managed safely through encapsulation. This task ties together inheritance, polymorphism, and encapsulation in a single integrated system, reflecting how these OOP principles work together in realistic AI-driven applications